¡Excelente! Ya que tenemos el Lexer funcionando perfectamente y limpio de errores, el siguiente gran paso es construir el AST (Árbol de Sintaxis Abstracta) y luego el Parser.

El Parser tomará la lista de tokens que genera tu Lexer y la convertirá en una estructura de objetos que tu programa pueda entender y ejecutar de forma lógica. Pero antes de escribir el Parser, necesitamos definir las clases de los objetos que formarán ese árbol.

A esto le llamamos Nodos del AST.

El Siguiente Paso: Definir los Nodos del AST (ast_nodes.py)

Vamos a crear el archivo src/language/ast_nodes.py. En este archivo definiremos dos grandes familias de clases:

1. **Expresiones (Expression): Todo lo que devuelve un valor (números, textos, operaciones matemáticas, variables, llamadas a funciones como see(x, y)).**
2. **Instrucciones (Instruction): Las líneas de comandos que ejecutan acciones o alteran el flujo (asignaciones, saltos goto, condiciones if, o acciones como move(...)).**

También crearemos una clase contenedora llamada CompiledProgram que guardará toda la información de una criatura compilada (su cabecera y sus instrucciones).

Crea el archivo src/language/ast_nodes.py

Copia este código en src/language/ast_nodes.py:

Python

```
from dataclasses import dataclass
from typing import List, Dict, Any, Optional


# ==============================================================================
# BASE
# ==============================================================================


class ASTNode:
    """Clase base para cualquier nodo del Árbol de Sintaxis Abstracta."""
    pass



# ==============================================================================
# EXPRESIONES (Devuelven un valor al evaluarse)
```

```
# ============================================================================

class Expression(ASTNode):
    """Clase abstracta para todas las expresiones."""
    pass


@dataclass
class LiteralInt(Expression):
    value: int

    def __repr__(self) -> str:
        return f"LiteralInt({self.value})"


@dataclass
class LiteralString(Expression):
    value: str

    def __repr__(self) -> str:
        return f"LiteralString({repr(self.value)})"


@dataclass
class VariableRef(Expression):
    """Representa el uso de una variable, percepción o constante (ej. health, x, NONE)."""
    name: str

    def __repr__(self) -> str:
        return f"VariableRef({self.name})"
```

```
@dataclass
class UnaryOp(Expression):
    """Operadores unarios: '-' (menos unario) y 'not'."""
    op: str  # "-" o "not"
    expr: Expression

    def __repr__(self) -> str:
        return f"UnaryOp({self.op}, {self.expr})"


@dataclass
class BinaryOp(Expression):
    """Operadores binarios: aritméticos (+,-,*,/,%), comparaciones (<,<=,>,>=,==,!=) y lógicos (and, or)."""
    op: str  # "+", "-", "and", "==", etc.
    left: Expression
    right: Expression

    def __repr__(self) -> str:
        return f"BinaryOp({self.left} {self.op} {self.right})"


@dataclass
class FunctionCall(Expression):
    """Llamadas a funciones del sistema (ej. see(x, y), name(x, y))."""
    name: str
    args: List[Expression]

    def __repr__(self) -> str:
```

```
        return f"FunctionCall({self.name}, {self.args})"


# ============================================================================
# INSTRUCCIONES (Realizan una acción o controlan el flujo)
# ============================================================================

class Instruction(ASTNode):
    """Clase abstracta para todas las instrucciones del cuerpo."""
    line_number: int


@dataclass
class LabelInst(Instruction):
    """Representa una etiqueta (ej. 'start:', 'flee:'). No consume turno."""
    name: str
    line_number: int

    def __repr__(self) -> str:
        return f"Label({self.name}:)"


@dataclass
class GotoInst(Instruction):
    """Salto incondicional (ej. 'goto start')."""
    target: str
    line_number: int

    def __repr__(self) -> str:
        return f"Goto({self.target})"
```

```
@dataclass
class ConditionalGotoInst(Instruction):
    """Salto condicional (ej. 'if health < 20 goto flee')."""
    condition: Expression
    target: str
    line_number: int

    def __repr__(self) -> str:
        return f"IfGoto({self.condition} -> {self.target})"


@dataclass
class AssignmentInst(Instruction):
    """Asignación de variable (ej. 'home_x = x')."""
    variable_name: str
    value_expr: Expression
    line_number: int

    def __repr__(self) -> str:
        return f"Assign({self.variable_name} = {self.value_expr})"


@dataclass
class ActionInst(Instruction):
    """Llamada a acción (ej. 'move(1, 0, 2)' o 'wait(1)')."""
    action_name: str
    args: List[Expression]
    line_number: int
```

```
    def __repr__(self) -> str:
        return f"Action({self.action_name}, {self.args})"


# ============================================================================
# PROGRAMA COMPILADO COMPLETO
# ============================================================================

class CompiledProgram:
    """Representa el resultado final de compilar un archivo .ins."""

    def __init__(self):
        # Cabecera de la criatura
        self.creature_name: str = ""
        self.faction: str = ""
        self.health: int = 0
        self.vision: int = 0
        self.lifespan: int = 0

        # Cuerpo
        self.instructions: List[Instruction] = []
        # Mapa de etiquetas a índices de instrucción para saltos ultra rápidos: {"start": 0, "flee": 14}
        self.labels: Dict[str, int] = {}

    def __repr__(self) -> str:
        return (
            f"CompiledProgram(\n"
            f"  Name: {self.creature_name}\n"
            f"  Faction: {self.faction}\n"
```

```
            f"  Health: {self.health}\n"
            f"  Vision: {self.vision}\n"
            f"  Lifespan: {self.lifespan}\n"
            f"  Instructions: {len(self.instructions)} lines\n"
            f"  Labels: {list(self.labels.keys())}\n"
            f")"
        )
```

¿Por qué lo hacemos así? (Diseño OOP de Excelencia)

Fíjate en lo limpio que queda:

1. **Cada tipo de instrucción y expresión tiene su propia clase especializada.**
2. **No usamos diccionarios genéricos ni tuplas sueltas para representar el código, sino objetos tipados. El compilador y el evaluador podrán usar polimorfismo o comprobar tipos fácilmente.**
3. **El CompiledProgram tiene un mapa de labels directo. Cuando una criatura haga un goto flee, el intérprete no tendrá que buscar línea por línea dónde está flee:. Solo mirará self.labels["flee"] y saltará instantáneamente al índice correspondiente en la lista de instrucciones. ¡Eficiencia pura!**